



הצעת פרויקט לעבודת גמר

יד הנדסאי תוכנה (שאלון: 714918)

בנושא

הצפנת מסרים בקבצים (סטגנוגרפיה)



שם סטודנט: יהונתן אליעזר עובדיה

ת"ז:

מנחה: אילן פרץ

תאריך: 13/01/2026

מכללה:

תוכן עניינים

1.	תיאור הנושא	3
2.	רקע תיאורטי	3
3.	תיאור הפרויקט	6
4.	הגדרת הבעיה האלגוריתמית	7
5.	הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות	12
6.	הליכים עיקריים בתחום למידת מכונה – לא רלוונטי	15
7.	הליכים עיקריים במע' הפעלה / רשתות / תקשורת / אבטחת – לא רלוונטי	15
8.	תיאור פרוטוקולי תקשורת	15
9.	פיתוחים עתידיים	16
10.	תיאור טכנולוגיה הנדסה	17
11.	מסד נתונים	18
12.	פרטים פורמליים	20

1. תיאור הנושא

תחום הסטגנוגרפיה (Steganography) הוא ענף מרכזי בעולם אבטחת המידע (Cyber Security) העוסק בטכנולוגיות להסתרת עצם קיומו של מידע. להבדיל מהצפנה קלאסית (Cryptography), אשר מערבלת את תוכן המידע אך מותירה אותו גלוי לעין כרצף תווים חסרי משמעות, מטרת הסטגנוגרפיה היא להטמיע מסרים בתוך מדיה דיגיטלית (כגון תמונות, קבצי שמע או וידאו) באופן שאינו מותיר עקבות גלויים, כך שמתבונן מהצד לא יחשוד כלל שמועבר מסר סמוי בתוך הקובץ.

מאפייני התחום והאתגרים הקיימים: בעולם אבטחת המידע המודרני, קיים צורך מבצעי בקיומם של ערוצי תקשורת חשאים לחלוטין עבור אוכלוסיות הפועלות בסביבה עוינת, כגון גורמי ביטחון או עיתונאים במדינות המפעילות צנזורה. השימוש בפתרונות הצפנה סטנדרטיים יוצר לעיתים קרובות "רעש" דיגיטלי בולט לעין, אשר עלול להסגיר את עצם קיומה של התקשורת ולהוביל לחסימתה. האתגר הטכנולוגי המרכזי בתחום זה הוא היכולת להחדיר מידע זר לתוך קובץ נתונים קיים ("קובץ כיסוי") בנפח משמעותי, מבלי לפגוע באיכותו הוויזואלית או השמיעתית של הקובץ, ומבלי לשנות את המאפיינים הסטטיסטיים שלו בצורה שתתגלה על ידי כלי ניטור ובקרה ברשת.

2. רקע תיאורטי

שיטות קיימות להטמעת מידע בקבצי תמונה ושמע:

1. LSB – שינוי הביט הפחות משמעותי

שיטה זו מבצעת החלפה של הביט האחרון בכל פיקסל בתמונה או בדגימת אודיו. היתרון המרכזי שלה הוא פשטות מימוש וקיבולת גבוהה: ניתן להטמיע כמות גדולה של מידע בקבצים גדולים יחסית, והעין האנושית בדרך כלל אינה מבחינה בשינויים זעירים בביט הפחות משמעותי.

עם זאת, החיסרון הגדול הוא פגיעות גבוהה: כל שינוי בסיסי בקובץ — דחיסה, המרת פורמט, שינוי בהירות, רעש, סיבוב או שמירה מחדש — עלול למחוק את המידע או לחשוף אותו. בנוסף, השיטה יוצרת תבניות סטטיסטיות שקל לזהות באמצעות כלים מודרניים. לכן, LSB אינה נחשבת לשיטה בטוחה בפרויקטים רציניים.

2. Histogram Shifting – הזזת התפלגות הפיקסלים

שיטה זו מבוססת על שינוי מינימלי של ההיסטוגרמה, כלומר התפלגות ערכי הפיקסלים בתמונה. האלגוריתם משתמש בכך כדי לפנות מקום למידע חדש מבלי ליצור עיוותים ברורים לעין. היתרון הוא חמקנות יחסית גבוהה: בדרך כלל השינויים קטנים מאוד והעין האנושית כמעט לא מבחינה בהם.

החיסרון הוא שהיכולת להטמיע מידע מוגבלת יחסית לשיטות אחרות. כל שינוי בעיבוד התמונה – כמו פילטר החלקה, שינוי קונטרסט או דחיסה – עלול לפגוע במסר המוטמע. לכן השיטה פחות מתאימה לתמונות שעשויות לעבור עריכה נוספת.

3. JSteg – החבאה במקדמי DCT של JPEG

JSteg פועל ישירות על מקדמי ה-DCT (הייצוג הפנימי של JPEG). היתרון המרכזי שלה היה אפשרות להחביא מידע מבלי לשנות את ערכי הפיקסלים עצמם. בשנות ה-90 ותחילת שנות ה-2000, היא הייתה אחת השיטות הפופולריות להטמעה בקבצי JPEG.

החולשה העיקרית היא יצירת חתימה סטטיסטית ברורה: הביטים משתנים במקדם DCT בצורה ניתנת לזיהוי. כיום, JSteg נחשבת מיושנת ומסוכנת לשימוש במקרים בהם נדרשת חמקנות או עמידות.

4. F3 ו-F4 – גרסאות מוקדמות של החבאה ב-JPEG

גרסאות אלה שיפרו מעט את JSteg כדי להפחית את הסימנים הסטטיסטיים. האלגוריתמים טיפלו בערכים חיוביים ושליילים בצורה שונה והקטינו במעט את הסיכון לגילוי המסר. למרות זאת, התוצאות אינן מספקות: שני האלגוריתמים עדיין משאירים תבניות שקל לזהות. F3 נוטה לאבד מידע כאשר מקדם הופך לאפס, ו-F4 מפחיתה את התבניות במעט בלבד. לכן הן אינן עמידות מספיק לשימוש בפרויקטים אמיתיים.

5. F5 – Matrix Encoding (גרסה מתקדמת של JSteg)

F5 היא אחת השיטות החזקות ביותר להטמעה ב-JPEG. השיטה משתמשת בקידוד מטריצוני שמאפשר להטמיע מידע רב תוך שינוי מינימלי של מקדמי DCT. האלגוריתם משנה מספר מועט של מקדמים במקום כל מקדם שבו יש מידע, מה שמקטין את החתימה הסטטיסטית ומעלה את החמקנות.

יתרון נוסף הוא שימוש בבחירה פסבדו-אקראית של המקדמים שבהם יוטמע המידע, מה שמקשה על גילוי סטטיסטי. השיטה ידועה כסטנדרטית בפרויקטים מחקריים ומאפשרת איזון טוב בין קיבולת, חמקנות ועמידות לדחיסה חוזרת.

6. PVD – Pixel Value Differencing (הבדלי פיקסלים)

PVD בוחנת זוגות פיקסלים סמוכים ומזהה את רמת השונות ביניהם. באזורים מורכבים כמו שיער, דשא, בניינים וטקסטורות — ניתן להטמיע הרבה מידע מבלי שאפשר להבחין בשינוי. באזורים חלקים כמו שמיים או קירות נקיים, האלגוריתם מטמיע מעט מידע בלבד כדי למנוע כתמים.

השיטה מנצלת את העובדה שהעין האנושית רגישה פחות לשינויים באזורים עמוסים בפרטים, ולכן מספקת חמקנות טובה מאוד. היא נחשבת לבטוחה יותר מ־LSB ומאפשרת קיבולת טובה עם איכות תמונה גבוהה.

7. Phase Coding – קידוד פאזה בקבצי שמע

Phase Coding מטמיע מידע בפאזה של גל הקול במקום לשנות עוצמה או תדר. היתרון הוא חמקנות טובה: השינוי כמעט בלתי מורגש לאוזן האנושית. החיסרון הוא שכל עיבוד על הקובץ — כמו פילטר, נורמליזציה או שינוי עוצמה — מוחק את המידע. לכן שיטה זו אינה עמידה מספיק לשימוש בקבצי שמע שעוברים עריכה או המרה.

8. Echo Hiding – הוספת הד עדין

שיטה זו מוסיפה הד חלש מאוד בתדרים מסוימים, שהאוזן שומעת כחלק מהאקוסטיקה. היתרון הוא חמקנות מסוימת והטמעה פשוטה.

החיסרון הוא שהד יוצר חתימה שקל לגלות באמצעות ניתוח ספקטרלי, וכל שינוי במבנה הקובץ, דחיסה או פילטרים, מוחק את המידע לחלוטין.

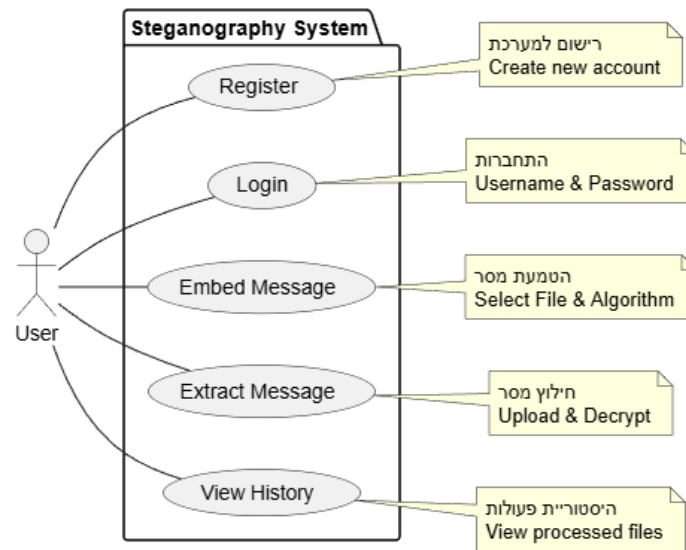
9. DSSS – Spread Spectrum (שיטה צבאית)

DSSS מפזר את המסר על פני ספקטרום רחב, כך שהמסר נשמע כמו רעש לבן טבעי. השיטה עמידה מאוד: כמעט כל שינוי בקובץ דחיסה, פילטרים, שינוי עוצמה או המרת פורמט אינו משפיע על המידע.

השיטה דורשת רצף פיזור שמוכר רק למי שמנסה לשחזר את המסר. DSSS משמשת במערכות צבאיות, לוויינים ותקשורת מוצפנת, ומספקת חמקנות ועמידות גבוהות יותר משאר השיטות.

3. תיאור הפרויקט

המערכת פועלת כאתר אינטרנט שמחביא הודעות כתובות בתוך קבצים כמו תמונות או שירים. הפעולה הזו גורמת לקבצים להיראות רגילים לגמרי למי שמסתכל מבחוץ, כך שהמסר נשמר בסוד. המערכת מעבירה את הקבצים הללו בין המשתמשים דרך תיבת דואר פנימית וסגורה, שומרת רישום מסודר של כל הפעולות שנעשו ביומן אישי, ועושה את כל זה ישירות דרך האינטרנט ללא צורך בהתקנת תוכנה.



להלן תרשים מקרי שימוש (Use Case Diagram)

- רישום (Register): יצירת חשבון משתמש חדש במערכת.
- התחברות (Login): כניסה למערכת באמצעות שם משתמש וסיסמה.
- הטמעת מסר (Embed): המשתמש מעלה קובץ "כיסוי" (תמונה/אודיו) ומקליד טקסט. המערכת מחזירה קובץ המכיל את הטקסט המוצפן.
- חילוץ מסר (Extract): העלאת קובץ המכיל מידע מוסתר וקבלת הטקסט המקורי ממנו.
- שליחה (Send): העברת הקובץ המוצפן בצורה מאובטחת למשתמש אחר בתוך המערכת.
- ניהול היסטוריה: צפייה בפעולות עבר והורדה חוזרת של קבצים שעובדו.

תרחיש דוגמה (Scenario): משתמש מעלה תמונת נוף רגילה למערכת ומקליד בתיבת הטקסט "ידיעה סודית". המערכת מטמיעה את הטקסט בתוך קובץ התמונה ושומרת אותו. המשתמש שולח את התמונה ה"תמימה" לחברו דרך המערכת, והחבר מוריד אותה ומחלץ מתוכה את המסר הסודי.

4. הגדרת הבעיה האלגוריתמית

האתגר המרכזי בפרויקט הוא להטמיע מסר בתוך קובץ קיים תוך עמידה בשני תנאים מנוגדים: שמירה על איכות הקובץ כך שהעין או האוזן האנושית לא ירגישו בשינוי, והבטחת עמידות כך שגורמים עוינים לא יצליחו לגלות או למחוק את המסר.

כדי לפתור את הבעיה הזו, המערכת לא עובדת בשיטה אחת, אלא משתמשת בפתרון שונה ("היברידי") לכל סוג קובץ, כדי להשיג את התוצאה הטובה ביותר.

ולכן ליבת הפתרון ההנדסי בפרויקט מבוססת על פיתוח שלושה מנועי עיבוד אותות ותמונה (Stego-Engines) נפרדים. המערכת משתמשת ברכיבי תוכנה סטנדרטיים לקריאת המבנה הבסיסי של הקבצים, ומממשת מעליהם את הלוגיקה הסטגנוגרפית הייחודית לכל פורמט. להלן פירוט ההליכים:

1. אלגוריתם – PVD (Pixel Value Differencing) לתמונות PNG

מדוע נבחר PVD ולא LSB או Histogram Shifting?

- PVD משנה רק אזורים מורכבים בפרטים, כך שהשינויים כמעט בלתי נראים
- מאפשר קיבולת גבוהה יותר מ-Histogram Shifting
- עמיד יותר מ-LSB: השינויים אינם ליניאריים ולכן קשה לגלות אותם בכלי סטגנליזה
- משמר איכות תמונה גבוהה גם כשמוסיפים מסרים ארוכים
- מתאים במיוחד ל-PNG שמירה על פיקסלים ללא דחיסה
- במקומות חלקים, השינויים כמעט אפסיים בזכות התאמה דינמית של האלגוריתם

כיצד פועל האלגוריתם:

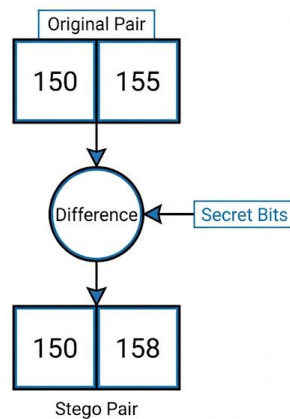
תהליך זה פועל במרחב המרחבי (Spatial Domain) ומנצל את רגישות העין האנושית לניגודיות (Contrast).

א. טעינה וחלוקה (Preprocessing) :

המערכת טוענת את התמונה למטריצת פיקסלים (Raster). האלגוריתם סורק את המטריצה ומחלק אותה לזוגות של פיקסלים סמוכים שאינם חופפים. עבור כל זוג פיקסלים (P_i, P_{i+1}) , מחושב ערך הפרש המתמטי ביניהם:

$$d = P_{i+1} - P_i$$

ערך זה (d) מהווה אינדיקטור לרמת ה"רעש" המקומי: הפרש נמוך מצביע על אזור חלק, והפרש גבוה מצביע על קצוות או טקסטורה.



איור 1: סכמת הטמעה בשיטת PVD האלגוריתם מחשב את הפרש (d) בין זוג פיקסלים, ומחליט על כמות המידע להטמעה בהתאם לטבלת הטווחים האדפטיבית.

ב. לוגיקה אדפטיבית (Range Table Logic) :

המערכת משתמשת בטבלת טווחים (R) שהוגדרה מראש. רוחב הטווח (w) שאליו משויך הפרש קובע את הקיבולת של הזוג בביטים, (n) לפי הנוסחה הלוגריתמית:

$$n = \text{floor}(\log_2(w))$$

הרציונל ההנדסי הוא שבאזורים רועשים (הפרש גבוה) ניתן להחביא מידע רב יותר מבלי שהעין תבחין בכך (מיסוך ויזואלי), בעוד שבאזורים חלקים ההטמעה היא מינימלית.

ג. הטמעה ועדכון (Embedding) :

האלגוריתם מחשב הפרש מטרה חדש (d') המכיל את המידע הסודי. לאחר מכן, זוג הפיקסלים המקורי מעודכן לערכים חדשים (P'_i, P'_{i+1}) על ידי הוספה או החסרה של מחצית מההפרש הדרוש:

$$m = |d' - d|$$

2. אלגוריתם – F5 (Matrix Encoding) לתמונות JPEG

מדוע נבחר F5 ולא F3, JSteg או F4?

- פועלת על מקדמי DCT בצורה חכמה ולא יוצרת תבניות שקל לגלות
- Matrix Encoding מצמצם את כמות השינויים הנדרשת
- עמידות גבוהה לדחיסה חוזרת של JPEG
- בחירה פסבדו-אקראית של מקדמים מונעת גילוי סטטיסטי
- מאפשרת קיבולת טובה תוך שמירה על איכות גבוהה
- נחשבת לסטנדרט מחקרי מודרני לשימוש בקבצי JPEG

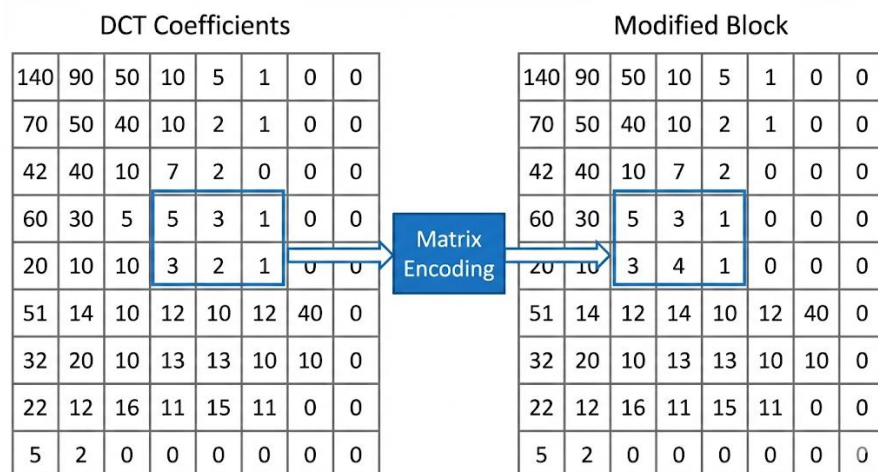
כיצד פועל האלגוריתם:

תהליך זה פועל במישור התדר (Frequency Domain) ונועד לשרוד דחיסה.

א. חילוץ מקדמים (Coefficient Extraction):

באמצעות שימוש בספרייט עיבוד תמונה מתאימה, המערכת מחלצת את מטריצת מקדמי ה-DCT (התדרים) של התמונה.

העבודה מתבצעת על המקדמים המקוונטים (Quantized Coefficients), שהם הייצוג המתמטי של התמונה לפני שלב הקידוד הסופי.



איור 2: ייצוג תמונה במישור התדר. העבודה מתבצעת על מטריצת מקדמי ה-DCT כאשר שינוי במקדם בודד משפיע על בלוק שלם בתמונה המפוענחת.

ב. ערבול וקידוד מטריצות (Permutation & Matrix Encoding) :

תחילה, המקדמים עוברים ערבול מבוסס מפתח לפיזור המידע.

לאחר מכן מופעלת ליבת האלגוריתם: שיטת קידוד המבוססת על קוד המינג $(1, k, n)$. שיטה זו מאפשרת להטמיע k ביטים בתוך קבוצה של מקדמים $(2^k - 1)$ על ידי שינוי הערך של מקדם אחד בלבד מתוך הקבוצה.

האלגוריתם מחשב XOR על מיקומי המקדמים, ואם התוצאה אינה תואמת למסר – הוא משנה מקדם יחיד כדי לתקן זאת. יעילות זו קריטית לשמירה על איכות התמונה.

ג. טיפול באפסים (Shrinkage Handling) :

האלגוריתם מזהה בזמן אמת מקרים שבהם שינוי מקדם גורם לו להתאפס (0). מכיוון שאפסים אינם נשמרים ביעילות ב-JPEG, המערכת מבצעת הטמעה חוזרת של אותו מידע בקבוצה הבאה.

3. אלגוריתם – DSSS (Spread Spectrum) לקבצי אודיו WAV

מדוע נבחר DSSS ולא Phase Coding או Echo Hiding?

- DSSS בשימוש צבאי ולווייני – אמינות גבוהה ומוכחת
- מפזר את המסר על כל הקובץ, כך שהשינויים כמעט בלתי נראים
- שינוי עוצמה, הוספת רעש, פילטר או דחיסה אינם מוחקים את המסר
- Phase Coding ו-Echo Hiding עלולות להימחק בקלות
- רק מי שמחזיק ברצף הפיזור יכול לשחזר את ההודעה
- נותנת שילוב טוב של חמקנות, עמידות וקיבולת

כיצד פועל האלגוריתם:

תהליך זה לקוח מעולם התקשורת ומיישם עקרונות של "רצף ישיר".

א. מחולל רעש (PN Generator) :

השלב הראשון בלוגיקה הוא יצירת "מפתח רעש". המערכת משתמשת בסיסמת המשתמש כדי לאתחל מחולל מספרים פסאודו-אקראיים, היוצר רצף ארוך (PN) של ערכים בינאריים (למשל +1 ו-1-).

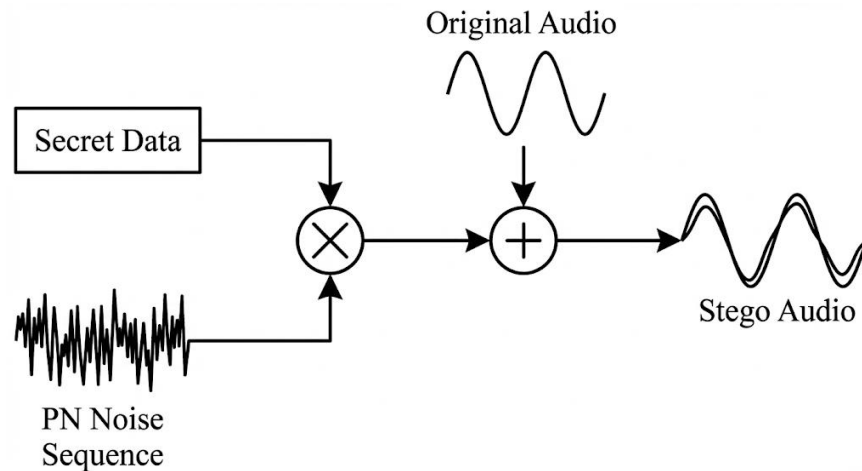
ב. אפנון והוספה (Modulation & Embedding) :

כל ביט של המסר הסודי "נמרח" על פני הרצף.

המידע מוכפל בפקטור הנחתה קטן מאוד, (α) כך שעוצמתו תהיה נמוכה מ"רצפת הרעש" של האודיו המקורי.

הנוסחה להוספת הרעש לדגימת האודיו (S) היא:

$$S' = S + (\alpha \cdot PN)$$



איור 3: מודל Spread Spectrum המידע מוכפל ברעש פסאודו־אקראי, (PN) מוחלש, ומתווסף לאות השמע המקורי כשכבה נסתרת.

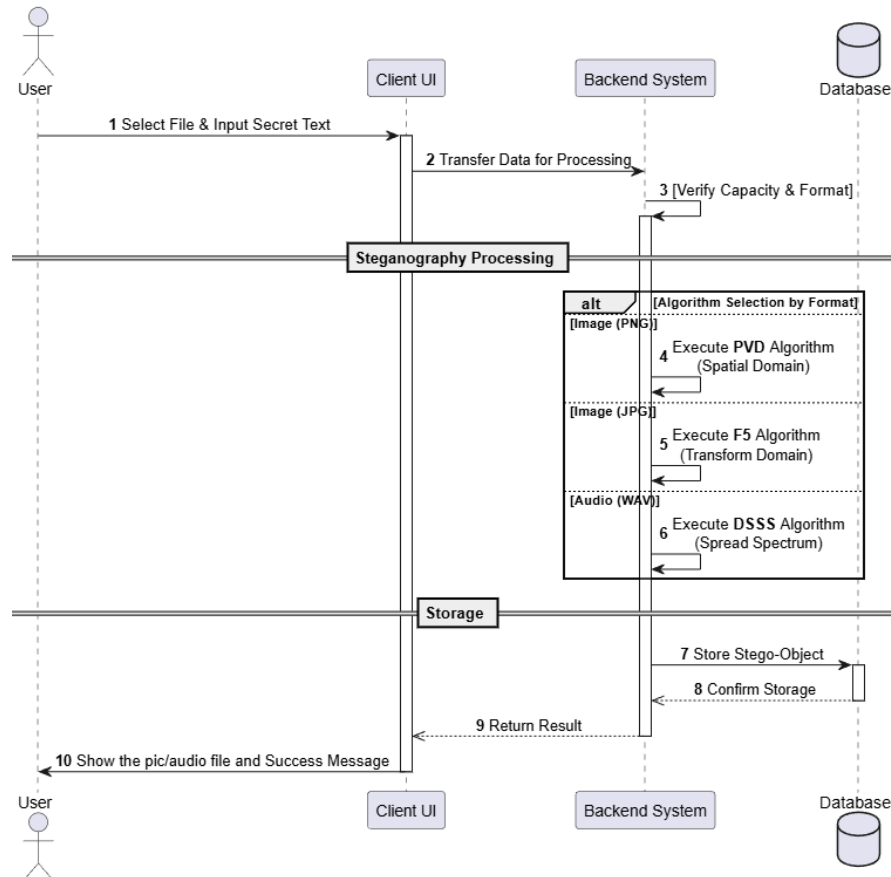
ג. חילוץ (Extraction) :

בצד המקבל, המערכת משתמשת באותו מפתח רעש ומבצעת חישוב מתמטי של קורלציה (Correlation) מול האודיו שהתקבל.

רעש הרקע האקראי מתבטל בחישוב, ורק התבנית של הרעש המוצפן "קופצת החוצה" ומאפשרת לשחזר את הביט המקורי.

5. הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות

להלן תרשים רצף (Sequence Diagram) המציג את זרימת המידע במערכת בעת ביצוע תהליך הסתרת המידע (Steganography). התרשים ממחיש את סדר הפעולות ואת האינטראקציה בין שכבת הלקוח (Client), צד השרת (Backend) ומסד הנתונים, החל משלב בחירת הקובץ ועד לשמירת התוצר המעובד. פירוט מלא של שלבי התהליך והאלגוריתמים השונים מופיע מיד לאחר התרשים.



התרשים מציג את רצף הפעולות הלוגיות מהקלט ועד להפקת התוצר הסופי:

1. בחירת קובץ והזנת טקסט סודי (Client UI): המשתמש בוחר קובץ (תמונה או אודיו) ומזין את המידע הסודי להטמעה.
2. העברת נתונים לעיבוד (Client → Backend): הקובץ והטקסט הסודי נשלחים למערכת השרת לצורך עיבוד.
3. אימות פורמט וקיבולת (Backend): המערכת בודקת את פורמט הקובץ והאם נפח המסר מתאים לקובץ הנבחר.

4. עיבוד סטגנוגרפי לפי סוג המדיה:

○ PNG -> ביצוע אלגוריתם PVD

○ JPG -> ביצוע אלגוריתם F5.

○ WAV -> ביצוע אלגוריתם DSSS.

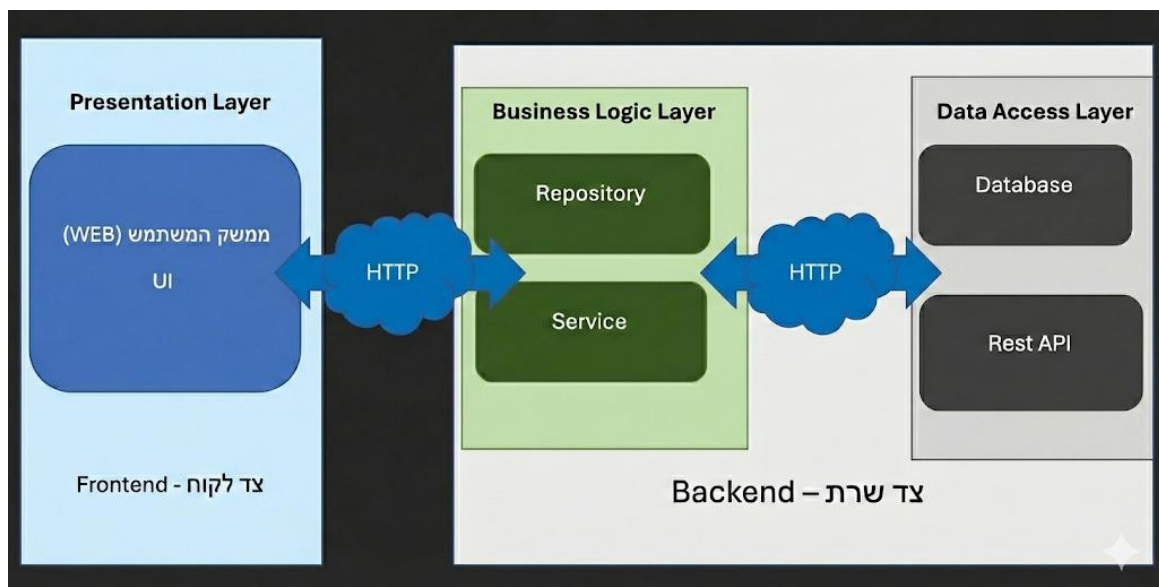
5. שמירה (Storage): לאחר הטמעת המידע, הקובץ המעובד נשמר בבסיס הנתונים.

6. אישור שמירה והחזרת תוצאה: השרת מאשר את שמירת הקובץ ושולח חזרה ללקוח את הקובץ המעובד.

7. הצגת תוצאה למשתמש (Client UI): המשתמש רואה את הקובץ המעובד יחד עם הודעת הצלחה.

ארכיטקטורת המערכת

הפתרון ההנדסי הינו מערכת **Web** הכוללת פיתוח **Full Stack** וממומשת באמצעות ארכיטקטורת "**שלושת השכבות**" (Three-Tier Architecture). אשר פועלת ע"פ מודל **שרת-לקוח** (Client-Server) על גבי רשת האינטרנט, ומודל זה מבטיח הפרדת רשויות (Separation of Concerns) ברורה בין ממשק המשתמש, הלוגיקה העסקית והנתונים.



מודל שרת-לקוח (Client-Server Model) הארכיטקטורה הכללית של המערכת מתבססת על מודל תקשורת של "בקשה ותגובה" (Request-Response). "במודל זה, הלקוח (הדפדפן) הוא הרכיב היוזם אשר שולח בקשות לקבלת שירותים או מידע, ואילו השרת הוא הרכיב המאזין, המעבד את הבקשות ומחזיר את הפלט הנדרש. יישום מודל זה במערכת הוא קריטי, שכן הוא מאפשר לרכז את משאבי העיבוד הכבדים כגון הרצת אלגוריתמי

הסטגנוגרפיה המורכבים וניהול אבטחת המידע בשרת המרכזי, תוך השארת צד הלקוח נגיש לשימוש מכל דפדפן סטנדרטי ללא תלות בחומרת הקצה של המשתמש.

1. שכבת התצוגה (Presentation Layer) - צד הלקוח (Client)

שכבת התצוגה היא הממשק מול המשתמש (Frontend) ומפותחת בטכנולוגיות Web מודרניות, כאשר היא פועלת בדפדפן של המשתמש ומספקת ממשק גרפי (GUI) לביצוע פעולות כגון רישום משתמשים, העלאת קבצים והצגת התוצרים. השכבה מבצעת ולידציה ראשונית של הקלט. היא שולחת בקשות HTTP או REST לשכבת הלוגיקה העסקית ומקבלת ממנה את התשובות המעובדות, מבלי לדעת כיצד הנתונים נשמרים או מעובדים.

2. שכבת הלוגיקה העסקית (Business Logic Layer) - צד השרת (Server)

שכבת הלוגיקה העסקית היא "המוח" של המערכת. היא ממומשת בשפת Java באמצעות Spring Boot, הכוללת שרת Apache Tomcat מובנה שמנהל את חיי האפליקציה, את תקשורת HTTP ניהול Threads ותלויות. השכבה אחראית על יישום האלגוריתמים המרכזיים כגון:

- מנוע סטגנוגרפי לתמונות PVD ל-PNG ו-F5 ל-JPEG.
- מנוע סטגנוגרפי לאודיו DSSS ל-WAV.
- מודלים לניהול מערכת ובקר גישה.

לשכבה זו יש תקשורת רציפה עם שכבת הנתונים באמצעות Repositories, שהיא שכבת ביניים המייצגת את הנתונים ומבודדת את הלוגיקה העסקית מהפרטים הטכניים של אחסון הנתונים. ה- Repositories מאפשרים ביצוע כל פעולות CRUD יצירה (Create), קריאה (Read), עדכון (Update), ומחיקה (Delete) מול בסיס הנתונים בענן. השימוש ב- Repositories מבטיח שהלוגיקה העסקית תישאר אותו דבר גם אם בסיס הנתונים ישתנה או יתעדכן.

3. שכבת הנתונים (Data Access Layer)

שכבת הנתונים אחראית על אחסון ושליפת מידע ניהולי ממסד נתונים מסוג NoSQL בענן כדוגמת MongoDB. השכבה מספקת את הפונקציות שמאפשרות Repositories לבצע CRUD בצורה בטוחה ויעילה, תוך בידוד שאר המערכת מהפרטים הטכניים של אחסון הנתונים. גישה זו מבטיחה אחידות ושקיפות בין השכבות, מאפשרת תחזוקה פשוטה של המערכת ומונעת תלות חזקה בין ה-Frontend וה-Backend ובסיס הנתונים.

6. הליכים עיקריים בתחום למידת מכונה – לא רלוונטי

הצעה זו לא עוסקת בלמידת מכונה, לכן סעיף זה לא רלוונטי.

7. הליכים עיקריים במע' הפעלה / רשתות / תקשורת / אבטחת – לא רלוונטי

הצעה זו לא עוסקת במע' הפעלה / רשתות / תקשורת / אבטחת מידע. לכן סעיף זה לא רלוונטי.

הערה חשובה: ההתייחסות לשרת לקוח מופיעה בהרחבה בסעיף 5 של ההצעה.

8. תיאור פרוטוקולי תקשורת

- **HTTP:** הוא פרוטוקול תקשורת ברשת שמאפשר העברת בקשות ותשובות בין מחשבים HTTPS. הוא הגרסה המאובטחת שלו, שמצפינה את הנתונים כדי למנוע קריאה או שינוי על ידי צדדים שלישיים. פרוטוקול זה נפוץ מאוד בשימוש ברשת האינטרנט ומאפשר תקשורת עקבית ומובנית בין שרתים ללקוחות.
- **REST API** הוא סגנון ארכיטקטוני לפרוטוקולי HTTP שבו כל משאב מזהה באמצעות כתובת ייחודית (URL) והגישה אליו נעשית באמצעות פעולות סטנדרטיות כגון GET, PUT, POST ו-DELETE. REST API מספק דרך עקבית ומודולרית לתקשורת בין רכיבים שונים במערכת, מאפשר שימוש חוזר בכתובות משאבים ומקנה יכולת הרחבה פשוטה של השירותים ברשת.
- **JSON:** הוא פורמט העברת נתונים פשוט, קריא ומבנה, המאפשר העברת מטא-דאטה, פרמטרים שונים של אלגוריתמים, הודעות מערכת, רשימות אוספים או תוצאות עיבוד בין הלקוח לשרת. השימוש ב-JSON מקל על פיתוח המערכת מכיוון שכל שפה מודרנית תומכת בפענוח והמרת JSON לאובייקטים פנימיים, ומבטיח אחידות בין כל חלקי המערכת. יתרון נוסף הוא שילוב קל עם REST API, שמאפשר שליחה וקבלה של נתונים בצורה מהירה ויעילה.
- **TCP/IP:** הוא מודל התקשורת שמאפשר העברת המידע ברשת בצורה אמינה ומדויקת. TCP אחראי על חלוקת המידע לחבילות, בדיקת שלמותן ושחזור סדר ההגעה שלהן.

בצד המקבל, מה שקריטי במיוחד במערכות סטגנוגרפיות שבהן חבילה אחת שלא הגיעה או הגיעה בסדר שגוי עלולה לשבש את הקובץ הבינארי ולמנוע חילוץ המסר IP. אחראי על ניתוב החבילות מהרכיב השולח אל הרכיב המקבל ברשת הגלובלית ומוודא שהמידע מגיע לכתובת הנכונה בצורה אופטימלית. השילוב TCP/IP מבטיח העברה אמינה ומדויקת של קבצים בינאריים, מסרים סטגנוגרפיים ומידע קריטי גם ברשתות מורכבות או לא יציבות.

השימוש בפרוטוקולים אלו מאפשר העברה אמינה ומדויקת של קבצים בינאריים בין הלקוח לשרת, מבטיח שהמידע המתקבל יהיה עקבי, שלם ומוכן לעיבוד על ידי האלגוריתמים הסטגנוגרפיים, מאפשר ניהול יעיל של פעולות CRUD בענן, תוך שמירה על תקשורת בטוחה ומהירה, ומספק בסיס איתן להרחבות עתידיות של המערכת, כולל שילוב רכיבי עיבוד נוספים או אבטחה משופרת.

9. פיתוחים עתידיים

1. שכבת הצפנה מקדימה (Pre-Encryption): הוספת שלב של הצפנה קריפטוגרפית (כגון AES) לטקסט, הממוקם לפני תהליך ההטמעה הסטגנוגרפית. שלב זה ישמש כשכבת אבטחה שנייה ("הגנה לעומק"), המבטיחה שגם אם גורם עוין יצליח לחלץ את המידע מתוך הקובץ, תוכנו יישאר בלתי קריא ללא מפתח מתאים.
2. הרחבה לפורמטי מדיה נוספים: הרחבת יכולות המערכת לתמיכה בסוגי קבצים דיגיטליים נוספים מעבר לתמונה ואודיו סטנדרטיים. פיתוח זה יאפשר לנצל מאפיינים ייחודיים של פורמטים שונים (כגון יתירות מידע בקבצי ארכיון או מסמכים) כדי לגוון את אפשרויות ההסתרה ולהגדיל את הקיבולת.
3. פיתוח אפליקציית מובייל (Mobile Application): בניית יישום לסמארטפונים. פיתוח זה ינגיש את המערכת לשימוש מידי בשטח, ויאפשר למשתמשים להצפין ולפענח קבצים (כגון תמונות שצולמו במכשיר) ישירות מהנייד ללא תלות בגישה לאינטרנט.
4. תוסף דפדפן (Browser Extension): פיתוח רכיב צד-לקוח המתממשק ישירות לדפדפן. תוסף זה יאפשר למשתמשים לנסות ולפענח מסרים מתמונות או קבצי אודיו בזמן אמת במהלך גלישה באתרי אינטרנט שונים, ללא צורך בהורדת הקובץ והעלאתו הידנית למערכת.

5. ניהול הודעות מתקדם (Advanced Messaging): שדרוג תיבת הדואר הפנימית כך שתכלול חיווי סטטוס קריאה.

10. תיאור טכנולוגיה הנדסה

שפת תכנות:

Java 21: היא שפת תכנות עילית OPP (מונחית עצמים). השפה מאפשרת כתיבה ברמת הפשטה גבוהה תוך שמירה על שליטה במבני נתונים, בזיכרון ובזרימת המידע Java. תומכת בעבודה עם טיפוסים פרימיטיביים, מערכי Bytes, מחלקות, ירושה וממשקים, ומשמשת לפיתוח מערכות תוכנה יציבות ומורכבות.

השימוש ב-Java מאפשר עבודה מדויקת עם נתונים בינאריים, אחידות טכנולוגית בין רכיבי מערכת שונים, ותמיכה בעיבוד מקבילי (Multithreading). סביבת ה-JVM-מספקת יציבות, ניהול זיכרון אוטומטי וביצועים עקביים, ולכן השפה מתאימה במיוחד לפיתוח מערכות הדורשות אמינות, שליטה ודיוק חישובי.

מסגרת עבודה (Framework):

- **Spring Boot 3.5.9:** משמשת כמסגרת העבודה המרכזית לניהול חיי האפליקציה, הלוגיקה והתקשורת בצד השרת. המסגרת כוללת שרת אפליקציה מובנה מסוג Apache Tomcat, האחראי לניהול בקשות HTTP ותקשורת בין הלקוח לשרת ללא צורך בשרת חיצוני Spring Boot. מאפשר הפרדה ברורה בין שכבת התקשורת לשכבת הליבה האלגוריתמית ותומך בניהול תהליכים.

- **Vaadin 24.9.9:** משמשת כמסגרת עבודה (Web Framework) לבניית ממשקי משתמש עשירים ומודרניים. המסגרת **מיועדת ומותאמת במיוחד למפתחי Java** ומאפשרת פיתוח Full-Stack בשפה אחידה: בניית רכיבי UI ולוגיקה בצד הלקוח באמצעות קוד Java בלבד, ללא צורך בכתיבה ישירה של HTML או JavaScript. גישה זו מבטיחה רצף טכנולוגי מלא, Type-Safety, ומצמצמת משמעותית סיכונים לבעיות המרה (Encoding) בעת העברת המידע בין השרת ללקוח.

ספריות עזר:

- **ספריות עזר לעיבוד תמונה:** המערכת עושה שימוש בספריות עזר סטנדרטיות של Java לעיבוד תמונה, המאפשרות גישה לנתוני פיקסלים, ביצוע חישובים ועיבודים

ברמת, Low-Level ויישום מניפולציות על קבצי תמונה ומדיה. ספריות אלו מאפשרות עבודה מדויקת עם הנתונים ושמירה על שליטה מלאה על הזרם הבינארי.

- **ספריות עזר לעיבוד קול וסאונד:** המערכת משתמשת בספריות עזר סטנדרטיות לעיבוד אותות קוליים, המאפשרות גישה לדגימות קול ועיבוד אותות ברמת-Low Level. שימוש בספריות אלו מאפשר יישום מניפולציות מדויקות על האות, עיבוד ושיפור איכות הקול, ושמירה על אמינות הנתונים.

סביבת פיתוח (IDE):

VS Code 1.107.1: סביבת פיתוח קלה ונוחה המותאמת לעבודה עם Java ו-Maven-באמצעות תוספים ייעודיים. הסביבה מאפשרת כתיבת קוד יעילה ואיתור שגיאות (Debugging) בזמן אמת, כולל אפשרות לעצור את התוכנה ולבדוק את הערכים השמורים בזיכרון. יכולת זו חיונית למעקב מדויק אחרי עיבוד הנתונים והקבצים במערכת. בנוסף, הסביבה מתחברת באופן מלא ל-Spring Boot ומאפשרת להפעיל ולנהל את השרת בקלות ישירות מתוך כלי הפיתוח.

11. מסד נתונים

ארכיטקטורת המערכת תתבסס על בסיס נתונים מסוג NoSQL המנוהל בענן, (Cloud) כדוגמת MongoDB אשר ישמש כתשתית האחסון המרכזית. תשתית זו תשמש לביצוע כלל פעולות ה-CRUD (יצירה, קריאה, עדכון ומחיקה), כאשר ייצוג הנתונים והעברתם בין רכיבי המערכת יתבצעו בפורמט JSON הסטנדרטי.

הבחירה בטכנולוגיית NoSQL נובעת מהצורך בגמישות מרבית במבנה הנתונים, בניגוד למסדי נתונים רלציוניים (SQL) המחייבים סכימה קשיחה. גמישות זו קריטית במיוחד עבור מערכת המנהלת אובייקטים ולוגים שעשויים להשתנות מעת לעת. בנוסף, ההחלטה להשתמש בשירות מנוהל בענן מבטיחה זמינות גבוהה (High Availability) וגישה רציפה מכל מקום, תוך ביטול הצורך בתחזוקה מורכבת של שרתים פיזיים.

להלן האוספים:

אוסף משתמשים (Users Collection):

- מזהה משתמש ייחודי (User ID).
- שם משתמש (Username).
- סיסמה (Password).

- פרטים נוספים: תאריך הצטרפות וסטטוס.

אוסף היסטוריית פעולות וקבצים (History & Files Collection):

אוסף זה מאפשר למשתמש לצפות בפעולות עבר ולהוריד קבצים.

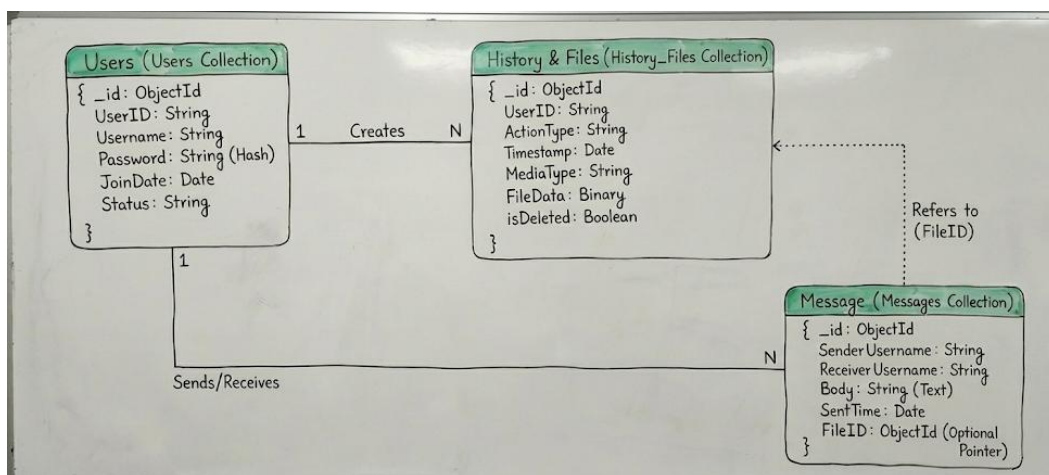
- מטא-דאטה: חותמת זמן, סוג הפעולה, וסוג המדיה.
- אחסון הקובץ: (Binary Data)** הקובץ הבינארי נשמר ישירות בתוך מסד הנתונים.
- סטטוס מחיקה: (isDeleted)** שדה בוליאני לביצוע "מחיקה לוגית", כך שהקובץ מוסתר מהמשתמש שמחק אך נשמר פיזית בשרת כל עוד הוא מקושר להודעות אחרות.

אוסף הודעות / תיבת דואר (Messages Collection):

אוסף זה תומך בשיתוף הקבצים והודעות טקסט בין משתמשים.

- שם השולח (Sender Username).
- שם המקבל (Receiver Username).
- תוכן ההודעה (Message Body).
- מזהה הקובץ (File ID): שדה אופציונלי. הפניה לקובץ בתוך הדאטה בייס (במידה וצורף קובץ להודעה).
- זמן שליחה (Sent Timestamp).

התרשים להלן המייצג את הקשרים בין האוספים:



12. פרטים פורמליים

לוח זמנים

לסיים עד תאריך	שלבי עבודה
1.12.2025	בחירת פרויקט, חקירה ולמידה לעומק של נושאי הפרויקט
11.12.2025	כתיבה והגשת הצעת הפרויקט לאישור משרד החינוך
13.1.2026	מימוש הקוד של האלגוריתם המרכזי, ביצוע בדיקות ושיפורים
3.2.2026	בניית צד שרת
17.2.2026	בניית מסד הנתונים ושילובו
3.3.2026	בניית צד לקוח
24.3.2026	כתיבת ספר הפרויקט
7.4.2026	הגשת הפרויקט כולו (ספר + קוד) להגנה וקבלת ציון מגן

חתימת מנחה הפרויקט: _____

חתימת הסטודנט _____

חתימת רכז המגמה: _____